Nesne Yönelimli Programlama

Hafta 6: Yapılar (struct), Birlikler (union) ve Numaralandırmalar

Marmara Üniversitesi • Mekatronik Mühendisliği Bölümü

Terim: Yapı (struct), heterojen veri türlerini tek bir çatı altında toplayan kullanıcı tanımlı birleşik veri tipidir.

Ders İzlencesi ve Temel Kazanımlar

Bu derste C++ programlama dilinde donanım seviyesi veri modellemenin temelleri incelenecektir:

- **Yapılar (`struct`):** Tanımlama, üye erişimi, yapı işaretçileri ve iç içe yapılar.
- **Yapılarda Metotlar:** Yapıcı (`constructor`) ve yıkıcı (`destructor`) kullanımı.
- `struct` **vs** `class`: Bellek modelleri ve varsayılan erişim belirleyicileri.
- **Birlikler (`union`):** Bellek paylaşımı ve ortak tampon yönetimi.
- **Bit Alanları (*Bit-fields*):** Donanım kaydedici ve bayt içi bit paketleme.
- **Bellek Hizalama ve Dolgu:** *Memory alignment*, *padding* ve `#pragma pack`.
- **Numaralandırmalar:** Geleneksel `enum` vs modern tür güvenli `enum class`.
- **Mekatronik Uygulaması:** CAN Bus telemetri protokolü veri çerçevesi tasarımı.

> **Terim:** Bellek Hizalama (Memory Alignment), işlemcinin verilere en yüksek hızda erişebilmesi için verileri belirli bayt sınırlarına yerleştirmesidir.

Prosedürel Veriden Bileşik Veriye: struct Felsefesi

Farklı türlerdeki sensör ve motor parametrelerini bağımsız değişkenlerde tutmak veri bütünlüğünü bozar:

- Birbirine bağlı değişkenlerin ayrı ayrı yönetilmesi yazılım karmaşıklığını artırır.
- Fonksiyonlara onlarca ayrı parametre aktarmak çağrı yığını (*call stack*) maliyetini yükseltir.
- `struct`, heterojen verileri mantıksal bir veri tipi halinde gruplar.
- C dilinde yalnızca veri tutan yapılar, C++ ile birlikte metot barındıran nesnelere dönüşmüştür.

```
1   // Bagimsiz degiskenler (Karmasik ve hataya acik)
2   int motorId = 1;
3   double motorHizi = 1500.0;
4   bool motorAktif = true;
5
6   // Bilesik yapi (Duzenli, butunlesik ve moduler)
7   struct MotorSurucu {
8       int id;
9       double hiz;
10      bool aktif;
11  };
```

Yapı Tanımlama ve Nesne Oluşturma

Yapı bildirimi yeni bir veri türü şablonu tanımlar; nesne bildirimi ise fiziki bellekte yer tahsis eder:

- `struct` anahtar kelimesi ile tür bildirimi yapılır ve noktalı virgül (`;`) ile sonlandırılır.
- Üye değişkenlere doğrudan erişim nokta (`.`) operatörü ile gerçekleştirilir.
- C++ dilinde değişken tanımlarken C'deki gibi `struct` anahtar kelimesinin tekrarlanması zorunlu değildir.

```
1  struct SensorVerisi {
2      int kanalNo;
3      double gerilim;
4      double sicaklik;
5  };
6
7  int main() {
8      SensorVerisi s1; // Nesne olusturuldu (Stack uzerinde tahsis)
9      s1.kanalNo = 1;
10     s1.gerilim = 3.32;
11     s1.sicaklik = 42.5;
12
13     SensorVerisi s2 = {2, 4.95, 28.1}; // Suslu parantez ile ilk deger atama
14     return 0;
```

Yapı Dizileri (Array of Structs)

Aynı yapı tipinden birden fazla elemanı ardışıl bellekte depolamak için yapı dizileri kullanılır:

- Çok kanallı telemetri sistemlerinde her sensör aynı yapı kalıbı ile temsil edilir.
- Dizi elemanları bellekte kesintisiz bir blok halinde sıralanır.
- İndisleme ve nokta operatörü bir arada kullanılır: `dizi[i].uye` .

```cpp
#include <iostream>

struct Eyleyici {
    int id;
    int hedefAci; // Derece (0 - 180)
};

int main() {
    const int ADET = 3;
    Eyleyici robotBilek[ADET] = {{1, 45}, {2, 90}, {3, 135}};

    for (int i = 0; i < ADET; ++i) {
        std::cout << "Motor " << robotBilek[i].id
                  << " -> Hedef Aci: " << robotBilek[i].hedefAci << " derece\n";
```

İç İçe Yapılar (Nested Structs)

Karmaşık mekatronik sistemler, daha küçük yapı bileşenlerinin birleştirilmesiyle hiyerarşik modellenir:

- Bir yapının üyesi, önceden tanımlanmış başka bir yapı türünde olabilir.
- Çok katmanlı erişim zincirleme nokta operatörü ile yapılır: `nesne.altYapi.uye` .
- Modülerlik, kod tekrarını önler ve yapıların farklı modüllerde yeniden kullanımını sağlar.

```
1   struct Konum3D {
2       double x;
3       double y;
4       double z;
5   };
6
7   struct RobotKolEklem {
8       int eklemId;
9       Konum3D koordinat; // Ic ice yapi kullanimi
10      double torkLimiti;
11  };
12
13  int main() {
14      RobotKolEklem eklem1 = {101, {12.5, -4.2, 85.0}, 45.0};
```

Yapı İşaretçileri ve Ok (->) Operatörü

Yapılara işaretçi aracılığıyla erişildiğinde ok (`->`) üye erişim operatörü kullanılır:

- `ptr->uye` ifadesi `(*ptr).uye` yazımının doğrudan eşdeğeridir.
- Operatör önceliği nedeniyle `*ptr.uye` yazımı hatalıdır (`.` operatörü `*` operatöründen önceliklidir).
- Fonksiyonlara büyük yapıları kopyalamadan aktarmak için yapı göstericileri veya referansları kullanılır.

```
1  struct TelemetriPaketi {
2      unsigned int zamanDamgasi;
3      float bataryaGerilimi;
4  };
5
6  void telemetriYazdir(const TelemetriPaketi* ptr) {
7      if (ptr != nullptr) {
8          std::cout << "Zaman   : " << ptr->zamanDamgasi << " ms\n";
9          std::cout << "Gerilim : " << ptr->bataryaGerilimi << " V\n";
10     }
11 }
```

Terim: Ok Operatörü (Arrow Operator - `->`), işaretçi üzerinden gösterilen nesnenin üyelerine dolaylı erişim sağlayan operatördür.

Dinamik Bellekte Yapı Tahsisi (new / delete)

Çalışma zamanında ihtiyaç duyulan yapı nesneleri Heap bölgesinde dinamik olarak tahsis edilir:

- `new` operatörü bellekten yapı boyutu kadar alan ayırır ve yapıcıyı çalıştırır.
- Ayrılan bellek mutlaka `delete` operatörü ile iade edilmelidir; aksi halde bellek sızıntısı oluşur.
- İşaretçi serbest bırakıldıktan sonra `nullptr` değerine çekilerek sarkan gösterici engellenir.

```
1  struct EnkoderVerisi {
2      long pulsSayaci;
3      int donusHizi; // RPM
4  };
5
6  int main() {
7      EnkoderVerisi* enkPtr = new EnkoderVerisi{1048576, 3000};
8
9      std::cout << "Puls: " << enkPtr->pulsSayaci << " | Hız: " << enkPtr->donusHizi << "\n"
10
11     delete enkPtr;       // Heap bellegi temizlendi
12     enkPtr = nullptr;    // Guvenli isaretci sifirlamasi
13     return 0;
14 }
```

struct vs class: Varsayılan Erişim Belirleyicisi

C++ dilinde `struct` ve `class` anahtar kelimeleri teknik olarak neredeyse tamamen özdeştir:

- **Erişim Farkı:** `struct` içinde üyeler varsayılan olarak `public`, `class` içinde ise `private` tır.
- **Kalıtım Farkı:** `struct` türetmelerinde varsayılan kalıtım modu `public`, `class` ta ise `private` tır.
- **Tasarım İlkesi:** Yalnızca veri tutan pasif yapılar için `struct`, kapsülleme ve invariant koruyan yapılar için `class` tercih edilir.

```
1  struct YapiNokta {
2      int x; // public (Varsayilan olarak dis erisime acik)
3  };
4
5  class SinifNokta {
6      int x; // private (Varsayilan olarak dis erisime kapali)
7  public:
8      void ayarla(int val) { x = val; }
9  };
```

Terim: Erişim Belirleyicisi (Access Specifier), sınıf veya yapı üyelerinin dış dünyadan doğrudan erişilebilirliğini kısıtlayan belirteçtir.

Yapılarda Kurucu Metot (Constructor)

Modern C++'ta yapılara kurucu metot tanımlanarak nesnelerin güvenli ilk değerlerle başlaması sağlanır:

- Kurucu metot, yapı ile aynı isme sahiptir ve geri dönüş tipi bulunmaz.
- Nesne tanımlandığı anda derleyici tarafından otomatik olarak çağrılır.
- Üye ilklendirme listesi (*member initializer list*) doğrudan bellek inşası sağlayarak performansı artırır.

```
1   struct DC_Motor {
2       int id;
3       int pwmPin;
4       double maksAkim;
5
6       // Uye ilklendirme listesi ile kurucu
7       DC_Motor(int _id, int _pin, double _akim)
8           : id(_id), pwmPin(_pin), maksAkim(_akim) {
9           // Kurucu govdesi gerekirse ek donanim denetimleri yapar
10      }
11  };
12
13  int main() {
14      DC_Motor m1(1, 9, 2.5); // Nesne guvenle ilklendirildi
```

Aşırı Yüklenmiş Kurucular (Constructor Overloading)

Farklı parametre kombinasyonları ile yapı nesnelerini başlatmak için kurucular aşırı yüklenir:

- **Varsayılan Kurucu:** Parametre almaz, üyeleri güvenli başlangıç değerlerine (sıfır, boş) çeker.
- **Parametreli Kurucu:** Dışarıdan sağlanan argümanlarla özel başlatma yapar.
- **Kopya Kurucu:** Mevcut bir nesnenin değerlerini yeni oluşturulan nesneye kopyalar.

```
1   struct KalmanDurumu {
2       double tahmin;
3       double hataKovaryansi;
4
5       KalmanDurumu() : tahmin(0.0), hataKovaryansi(1.0) {} // Varsayilan
6
7       KalmanDurumu(double t, double p) : tahmin(t), hataKovaryansi(p) {} // Parametreli
8
9       KalmanDurumu(const KalmanDurumu& diger) // Kopya Kurucu
10          : tahmin(diger.tahmin), hataKovaryansi(diger.hataKovaryansi) {}
11  };
```

Terim: Fonksiyon Aşırı Yükleme (Overloading), aynı isimli fonksiyonun farklı parametre sayısı veya türleriyle birden fazla kez tanımlanabilmesidir.

Yapılarda Yıkıcı Metot (Destructor) ve RAII

Yapı nesnesi kapsam dışına çıktığında (*out of scope*) veya `delete` edildiğinde yıkıcı çalışır:

- Yıkıcı metot yapı adı ile aynıdır ve başına tilde (`~`) simgesi alır.
- Parametre almaz ve geri dönüş değeri yoktur; aşırı yüklenemez.
- Dinamik bellek, açık donanım portları veya dosya tanıtıcılarının güvenle kapatılmasını sağlar.

```
1   #include <iostream>
2
3   struct TamponYonetici {
4       int* veriAlani;
5       int kapasite;
6
7       TamponYonetici(int k) : kapasite(k) {
8           veriAlani = new int[kapasite];
9       }
10
11      ~TamponYonetici() {
12          delete[] veriAlani; // Bellek sizintisini onleyen otomatik temizlik
13          std::cout << "Tampon bellegi basariyla iade edildi.\n";
14      }
```

Birlikler (Union) ve Bellek Paylaşımı İlkesi

Birlik (`union`), tüm üye değişkenlerin aynı fiziki bellek alanını paylaştığı özel bir veri türüdür:

- Yapılarda her üye için bellekte ayrı yer ayrılırken, birliklerde tek bir bellek havuzu bulunur.
- Birlik içindeki üyelerden belirli bir anda **yalnızca biri** aktif ve geçerli veriyi tutar.
- Bir üyeye değer yazıldığında, ortak bellek alanı güncellendiği için diğer üyelerin önceki verisi bozulur.
- Birlik boyutu, en büyük üyenin bellek boyutuna (ve hizalama katsayısına) eşittir.

```
union OrtakVeri {
    int tamSayi;     // 4 byte
    float ondalik;   // 4 byte
    char karakter;   // 1 byte
}; // Toplam bellek boyutu yalnizca 4 byte'tir
```

Terim: Birlik (Union), tüm üyeleri bellekte aynı başlangıç adresini paylaşan ve tek seferde tek bir üyesi geçerli olan veri yapısıdır.

struct ile union Bellek Mimarisi Karşılaştırması

İki veri yapısı arasındaki temel fark, üyelerin bellekteki yerleşim düzenidir:

- `struct` yapısında toplam boyut, üyelerin boyutlarının ve aradaki dolgu baytlarının toplamıdır.
- `union` yapısında toplam boyut, yalnızca en geniş üyenin kapladığı alan kadardır.

```
1  struct SensorStruct {
2      int id;         // 4 byte (Adres: 0x1000 - 0x1003)
3      double deger;   // 8 byte (Adres: 0x1008 - 0x100F) -> 4 byte dolgu var
4  }; // sizeof(SensorStruct) = 16 byte
5
6  union SensorUnion {
7      int id;         // 4 byte (Adres: 0x2000 - 0x2003)
8      double deger;   // 8 byte (Adres: 0x2000 - 0x2007)
9  }; // sizeof(SensorUnion) = 8 byte (Tum uyeler 0x2000 adresinden baslar)
```

Terim: Bellek Taban Çakışması, birlikteki tüm değişkenlerin adreslerinin aynı başlangıç baytına işaret etmesi durumudur.

Birliklerde Tek Aktif Üye Davranışı

Birlik üyeleri arasında geçiş yapıldığında bellekteki bayt dizilimi doğrudan değişir:

- Aktif olmayan bir üyeyi okumak derleyici seviyesinde tanımsız davranışa (*UB*) yol açabilir.
- Programcı, birlikteki hangi alanın o an geçerli olduğunu dikkatle takip etmelidir.

```cpp
#include <iostream>

union DonanimKaydi {
    int tamsayi;
    float gerilim;
};

int main() {
    DonanimKaydi reg;
    reg.tamsayi = 100;
    std::cout << "Tamsayi degeri : " << reg.tamsayi << "\n"; // Gecerli

    reg.gerilim = 3.3f; // tamsayi alaninin uzerine yazildi!
    std::cout << "Gerilim degeri : " << reg.gerilim << "\n"; // Gecerli
    std::cout << "Tamsayi bozuldu: " << reg.tamsayi << "\n"; // Bozuk bayt karsiligi
    return 0;
}
```

Birlikler ile Bayt Düzeyinde Veri Dönüşümü (Type Punning)

Birlikler, gömülü sistemlerde çok baytlı verileri tek tek baytlara ayırmak için sıkça kullanılır:

- 32-bitlik bir tam sayı veya kesirli telemetri verisi 4 adet 8-bitlik bayt dizisine bölünür.
- Seri iletişim protokollerinde (UART, SPI, I2C) veriler bayt bayt hatta aktarılır.
- Ortak bellek paylaşımı sayesinde herhangi bir matematiksel bölme veya kaydırma gerekmez.

```
union FloatDonusturucu {
    float veri;
    unsigned char baytlar[4];
};

FloatDonusturucu paket;
paket.veri = 45.75f;
// paket.baytlar[0] .. paket.baytlar[3] dogrudan UART TX tamponuna verilir
```

Terim: Type Punning, bellekteki aynı bit dizilimini farklı bir veri tipiymiş gibi doğrudan yorumlama tekniğidir.

Etiketli Birlik Tasarımı (Tagged / Discriminated Union)

Birliklerin hangi üyesinin aktif olduğunu güvenle bilmek için etiketli birlik deseni uygulanır:

- Birlik, bir `enum` türü belirteç ile birlikte bir `struct` içerisine sarılır.
- Dış yapı, hem veri yükünü (*payload*) hem de geçerli veri tipini (*type tag*) saklar.
- Bu yaklaşım modern C++17 `std::variant` konteynerinin temel çalışma mekanizmasıdır.

```
1   enum class PaketTipi { SICAKLIK, HIZ, ALARM };
2
3   struct GuvenliPaket {
4       PaketTipi tip; // Aktif uyeyi belirten etiket
5       union {
6           float sicaklik;
7           int motorHizi;
8           char hataKodu;
9       } veri; // Isimsiz birlik (Anonymous union)
10  };
```

Terim: Etiketli Birlik (Tagged Union), birliğin o anki geçerli üyesini bir tip etiketi ile izleyen güvenli veri yapısıdır.

Bit Alanları (Bit-fields) ve Donanım Modelleme

Mikrodenetleyici kontrol ve durum kayıtçıları bayt seviyesinde değil, bit seviyesinde yönetilir:

- Bit alanları, bir tamsayı üyenin bellekte kaç bit kaplayacağını açıkça belirlemeyi sağlar.
- `tip degisken_adi : bit_sayisi;` sözdizimi kullanılır.
- 1 baytlık (8-bit) tek bir hafıza alanında 8 farklı durum bayrağı paketlenebilir.

```
1  struct MotorKontrolKaydi {
2      unsigned char motorAcik     : 1; // 1 bit (0 veya 1)
3      unsigned char yonIleri      : 1; // 1 bit (0: Geri, 1: Ileri)
4      unsigned char frenAktif     : 1; // 1 bit
5      unsigned char hizKademesi   : 3; // 3 bit (0 - 7 arasi deger)
6      unsigned char hataKodu      : 2; // 2 bit (0 - 3 arasi hata)
7  }; // Toplam yalnizca 1 byte (8 bit)!
```

Terim: Bit Alanı (Bit-field), yapı üyelerine belirli sayıda bit genişliği tahsis ederek bellek tasarrufu sağlayan C++ özelliğidir.

Bit Alanları ile Donanım Kaydedici Yönetimi

Bit alanları sayesinde karmaşık bit maskeleme ve kaydırma operatörleri kullanma ihtiyacı azalır:

- Geleneksel bit işlemlerinde `REG |= (1 << 3)` gibi okunması zor maskeler kullanılır.
- Bit alanları ile değişken ataması yapar gibi doğrudan bitlere değer atanır: `reg.frenAktif = 1;` .
- Derleyici, arka planda gerekli bit maskeleme ve kaydırma talimatlarını otomatik üretir.

```
1   MotorKontrolKaydi motor1;
2   motor1.motorAcik = 1;
3   motor1.yonIleri = 1;
4   motor1.frenAktif = 0;
5   motor1.hizKademesi = 5; // 3 bitlik alana 5 (101_2) yazilir
6   motor1.hataKodu = 0;
7
8   if (motor1.hizKademesi > 4) {
9       // Hiz limiti denetimi
10  }
```

Terim: Bit Maskeleme (Bit Masking), bir kaydedicideki belirli bitleri `AND` , `OR` , `XOR` operatörleriyle izole etme veya değiştirme işlemidir.

Birlik ve Bit Alanı Hibrit Mimarisi

Gömülü sistemlerde kaydediciye hem bit bit hem de tek bir ham bayt olarak erişmek istenir:

- Bir `union` içerisinde hem bit alanı yapısı hem de tam sayı değişkeni tanımlanır.
- Böylece kaydediciye hem tek komutla ham bayt yazılabilir hem de tek tek bayraklar okunabilir.

```
union DurumKaydedici {
    unsigned char hamBayt; // Tum kaydediciye tek seferde erisim
    struct {
        unsigned char hazir     : 1;
        unsigned char iletisim  : 1;
        unsigned char asiriAkim : 1;
        unsigned char rezerve   : 5;
    } bitler;
};

DurumKaydedici reg;
reg.hamBayt = 0x05; // 00000101_2 yuklendi -> hazir=1, iletisim=0, asiriAkim=1
if (reg.bitler.asiriAkim) {
    // Acil durum korumasi
}
```

Terim: Hibrit Kaydedici (Hybrid Register Mapping), bit alanları ve ham tamsayının birlik içinde birleştirilerek donanım portuna

Bit Alanlarında Kısıtlamalar ve Dikkat Edilmesi Gerekenler

Bit alanları donanım programlamada büyük kolaylık sağlasa da bazı kritik sınırlamalara sahiptir:

- **Adres Alınamaz:** Bit alanları bayt sınırında bitmeyebileceğinden `&` adres operatörü uygulanamaz (`&reg.bitler.hazir` derleme hatası verir).
- **İşaretçilerle İşlem Yapılamaz:** Bir bit alanına doğrudan işaretçi bağlanamaz.
- **Bit Dizilimi (Endianness):** Bitlerin en anlamsız bitten (*LSB*) mi yoksa en anlamlı bitten (*MSB*) mi başlayacağı derleyiciye ve donanım mimarisine bağlıdır.
- Taşınabilir gömülü kütüphanelerde mimariye özel ön işlemci koşulları (`#ifdef`) kullanılır.

```
1  DurumKaydedici reg;
2  // int* ptr = &reg.bitler.hazir; // DERLEME HATASI: Bit alaninin adresi alinamaz!
```

Terim: Bayt Sıralaması (Endianness), çok baytlı verilerin bellekte düşük adresten yükseğe mi (Little-Endian) yoksa tersine mi (Big-Endian) dizileceğidir.

Bellek Hizalama (Memory Alignment) ve İşlemci Mimarisi

Modern mikroişlemciler verileri bellekte rastgele adreslerden değil, doğal sınırlarından okur:

- 32-bit (4-byte) bir `int` değişkenin adresi 4'ün katı olan bir bellek adresinde olmalıdır.
- 64-bit (8-byte) bir `double` değişkenin adresi 8'in katı olan bir adreste bulunmalıdır.
- Hizalanmamış (*unaligned*) bellek erişimi işlemcinin iki kat bellek çevrimi yapmasına veya istisna üretmesine yol açar.
- Derleyici, bu hizalama kuralını sağlamak için değişkenlerin arasına otomatik boşluklar ekler.

```
1  struct OrnekHizalama {
2      char a;    // 1 byte (Adres: 0x00)
3      // 3 BYTE DOLGU (PADDING) EKLENIR!
4      int b;     // 4 byte (Adres: 0x04)
5  }; // sizeof degeri 5 degil, 8 byte cikar!
```

Terim: Doğal Hizalama (Natural Alignment), bir veri tipinin bellekteki adresinin kendi bayt boyutunun tam katı olması zorunluluğudur.

Dolgu Baytları (Padding) ve Bellek İsrafı

Yapı üyelerinin tanımlanma sırası, derleyicinin ekleyeceği dolgu baytı miktarını doğrudan etkiler:

- Kötü sıralanmış bir yapıda gereksiz dolgu baytları nedeniyle bellek tüketimi iki katına çıkabilir.
- Yapı boyutu, yapının içindeki en büyük üyenin hizalama katına tamamlanır (*tail padding*).

```
1   // Verimsiz Yerlesim: 12 Byte
2   struct Verimsiz {
3       char c1;   // 1 byte + 3 byte dolgu
4       int i;     // 4 byte
5       char c2;   // 1 byte + 3 byte dolgu (son dolgu)
6   }; // Toplam: 12 byte!
7
8   // Optimize Edilmis Yerlesim: 8 Byte
9   struct Verimli {
10      int i;     // 4 byte
11      char c1;   // 1 byte
12      char c2;   // 1 byte + 2 byte dolgu
13  }; // Toplam: 8 byte! (Bellekten %33 tasarruf)
```

Terim: Dolgu Baytları (Struct Padding), üyelerin doğal hizalama sınırlarını karşılaması amacıyla derleyici tarafından eklenen kullanılmayan baytlardır.

Donanım İletişiminde Paketleme (#pragma pack)

Ağ üzerinden veya seri porttan (UART, SPI) gönderilen veri paketlerinde dolgu baytları istenmez:

- Karşı cihaz (örneğin 8-bit bir mikrodenetleyici) dolgu baytlarından habersiz veri bekler.
- `#pragma pack(push, 1)` direktifi derleyicinin tüm dolgu baytlarını sıfırlamasını sağlar.
- Bu işleme yapı paketleme (*struct packing*) denir ve veri paketinin saf halini korur.

```
#pragma pack(push, 1) // Dolguyu kapat (1-byte hizalama)
struct TelemetriCercevesi {
    char baslik;   // 1 byte
    int veri;      // 4 byte (Dolgu eklenmez, dogrudan 1. bayttan baslar)
    char sonlandir;// 1 byte
}; // sizeof tam olarak 1 + 4 + 1 = 6 byte cikar!
#pragma pack(pop)     // Onceki derleyici hizalama ayarina geri don
```

Terim: Yapı Paketleme (Struct Packing), derleyicinin dolgu eklemesini engelleyerek üyeleri bellekte bitişik sıkıştırma yöntemidir.

alignof ve alignas Standartları (C++11)

Modern C++ standartlarında hizalama gereksinimlerini sorgulamak ve yönetmek için anahtar kelimeler bulunur:

- `alignof(T)` : Bir veri tipinin bellekte kaç baytlık sınıra hizalanması gerektiğini verir.
- `alignas(N)` : Bir değişkenin veya yapının zorunlu olarak N baytlık sınırda hizalanmasını sağlar.
- DMA (Doğrudan Bellek Erişimi) tamponlarında 32 veya 64 baytlık özel önbellek hizalamaları gereklidir.

```cpp
#include <iostream>

struct alignas(16) OzelVektor {
    float x, y, z, w; // SIMD ve DMA islemleri icin 16-byte hizali blok
};

int main() {
    std::cout << "int hizalamasi     : " << alignof(int) << " byte\n";
    std::cout << "double hizalamasi  : " << alignof(double) << " byte\n";
    std::cout << "OzelVektor hizalama: " << alignof(OzelVektor) << " byte\n";
    return 0;
}
```

Sabit Değer Yönetimi: Geleneksel enum ve Kapsam Kirliliği

C tarzı klasik numaralandırmalar ilişkili sabitleri gruplamak için kullanılır ancak zayıf yönleri vardır:

- **Kapsam Kirliliği:** Enum üyeleri tanımlandıkları kapsamın genel isim alanına sızar.
- **Örtük Tür Dönüşümü:** Tamsayılara otomatik dönüşür, tür güvenliği (*type safety*) yoktur.
- Farklı iki enum aynı isimde bir sabite sahip olamaz (çakışma hatası).

```
1   enum MotorDurumu {
2       DURDU,   // Global kapsama sizdi
3       CALISIYOR,
4       HATA
5   };
6
7   enum SensorDurumu {
8       BEKLEMEDE,
9       // HATA // DERLEME HATASI: 'HATA' zaten MotorDurumu icinde tanimlandi!
10  };
11
12  int x = DURDU; // Guvensiz ortuk donusum: int degiskenine dogrudan atanabilir
```

Tür Güvenli Numaralandırmalar: enum class (C++11)

C++11 ile tanıtılan kapsamlı numaralandırmalar (*Scoped Enums*) geleneksel zafiyetleri çözer:

- `enum class` anahtar kelimesi ile tanımlanır.
- Üyelere erişmek için mutlaka kapsam operatörü (`::`) kullanılmalıdır: `MotorDurum::Hata` .
- Tamsayılara otomatik dönüşmez; `static_cast` olmaksızın tamsayı değişkenlerine atanamaz.

```
1   enum class MotorDurum {
2       Durdu,
3       Calisiyor,
4       Hata
5   };
6
7   enum class SensorDurum {
8       Beklemede,
9       Calisiyor, // Isim cakismasi yasanmaz! Kendi kapsamindadir
10      Hata
11  };
12
13  MotorDurum durum = MotorDurum::Calisiyor;
14  // int sayi = durum; // DERLEME HATASI! Guvenli tip denetimi devrededir.
```

enum class ile Taban Türün Belirlenmesi

Gömülü sistemlerde numaralandırmanın bellekte ne kadar yer tutacağını denetlemek kritiktir:

- Varsayılan olarak `enum class` mimarinin standart `int` türünü (genellikle 4 byte) kullanır.
- İki nokta (`:`) operatörü ile taban tür açıkça belirtilebilir: `uint8_t` , `int16_t` vb.
- Bu sayede telemetri paketlerinde 1 baytlık protokol durum kodları üretilebilir.

```cpp
#include <cstdint>

enum class HataKodu : uint8_t {
    HataYok         = 0x00,
    AsiriGerilim    = 0x01,
    AsiriSicaklik   = 0x02,
    IletisimKopuk   = 0x03
}; // sizeof(HataKodu) kesin olarak 1 byte'tir!

HataKodu err = HataKodu::AsiriSicaklik;
uint8_t hamKod = static_cast<uint8_t>(err); // Acik tur donusumu
```

Terim: Taban Tür (Underlying Type), bir numaralandırmanın bellekte saklanacağı tamsayı veri tipini belirleyen mekanizmadır.

Tür Takma Adları: typedef vs using

Karmaşık yapı ve işaretçi bildirimlerini sadeleştirmek için tür takma adları kullanılır:

- C dilinden gelen geleneksel `typedef` sözdizimsel olarak karmaşıklaşabilir.
- C++11 ile gelen `using` ifadesi, atama mantığına uygunluğu sayesinde daha okunaklıdır.
- Modern C++ standartlarında şablon uyumluluğu nedeniyle `using` kullanımı tavsiye edilir.

```
1  // Geleneksel C-tarzi typedef
2  typedef unsigned long long int ZamanDamgasi;
3
4  // Modern C++11 using bildirimi (Tercih edilen yaklasim)
5  using Milisaniye = unsigned long long int;
6  using SensorOkumaFonksiyonu = double (*)(int kanalNo);
7
8  Milisaniye anlikZaman = 1609459200000ULL;
```

Terim: Tür Takma Adı (Type Alias), mevcut bir veri tipine kod okunabilirliğini artırmak amacıyla verilen alternatif isimdir.

Mekatronik Vaka Analizi: CAN Bus Telemetri Mimarisi

Otomotiv ve endüstriyel robotik ağlarında veriler CAN Bus veri çerçeveleri ile iletilir:

- Standart bir CAN veri çerçevesi en fazla 8 bayt (*Payload*) veri taşır.
- Bu 8 baytın içinde motor hızı, sıcaklık, hata bayrakları ve mod bilgisi paketlenir.
- `struct`, `union`, `bit-fields` ve `enum class` sentezlenerek sıfır kopyalama ile paketleme yapılır.

```
+-----------------------------------------------------------+
|                   CAN 8-Byte Veri Yükü                    |
+---------------+---------------+---------------+-----------+
| Bayt 0 - 1    | Bayt 2        | Bayt 3        | Bayt 4 - 7    |
+---------------+---------------+---------------+-----------+
| Hiz (int16_t) | Sicaklik (u8) | Bit Bayraklari| Konum (float) |
+---------------+---------------+---------------+-----------+
```

Terim: CAN Bus (Controller Area Network), mikrodenetleyicilerin ana bilgisayar olmadan haberleşmesini sağlayan dayanıklı endüstriyel veri yolu standardıdır.

CAN Bus Çerçevesinin Kod Mimarisi

Gömülü C++ yazılımında donanım veri çerçevesi şu şekilde modellenir:

```cpp
#include <cstdint>

enum class SurucuModu : uint8_t { MANUEL = 0, OTONOM = 1, ACIL_STOP = 2 };

#pragma pack(push, 1)
struct CanTelemetri {
    int16_t motorHizi;        // 2 byte
    uint8_t govdeSicakligi;   // 1 byte
    struct {
        uint8_t frenDevrede : 1; // 1 bit
        uint8_t asiriYuk    : 1; // 1 bit
        uint8_t rezerve     : 6; // 6 bit
    } durumBayraklari;          // 1 byte
    SurucuModu calismaModu;   // 1 byte
    float eklemPozisyonu;     // 4 byte
}; // Toplam: Tam 9 byte (Hizali ve sifir dolgulu paket)
#pragma pack(pop)
```

Terim: Protokol Çerçevesi (Protocol Frame), haberleşme hattında taşınan yapılandırılmış bayt dizisinin biçimsel tanımıdır.

CAN Bus Veri Paketleme ve Çözümleme

Birlik kullanılarak donanım seviyesinde ham baytlar doğrudan çerçeveye dönüştürülür:

```cpp
#include <iostream>

union CanCerceveTamponu {
    CanTelemetri telemetri;
    uint8_t hamBaytlar[sizeof(CanTelemetri)];
};

int main() {
    CanCerceveTamponu paket;
    paket.telemetri.motorHizi = 2450;
    paket.telemetri.govdeSicakligi = 55;
    paket.telemetri.durumBayraklari.frenDevrede = 0;
    paket.telemetri.durumBayraklari.asiriYuk = 1;
    paket.telemetri.calismaModu = SurucuModu::OTONOM;
    paket.telemetri.eklemPozisyonu = 124.5f;

    std::cout << "Paket Boyutu: " << sizeof(paket) << " bayt\n";
    std::cout << "CAN Veri Yolu Hatti Icin Baytlar Hazirlandi.\n";
    return 0;
}
```

Mekatronik Durum Makinesi ve enum class Entegrasyonu

Endüstriyel robot denetiminde durum makinesi geçişleri `enum class` ile yönetilir:

- Hatalı durum atamaları derleme zamanında engellenir.
- `switch-case` bloklarında tüm durumların ele alınıp alınmadığı derleyici uyarıları ile denetlenir.

```
enum class RobotDurum : uint8_t { INIT, BEKLEME, CALISMA, HATA_DURUMU };

void durumMakinesiniGuncelle(RobotDurum suankiDurum) {
    switch (suankiDurum) {
        case RobotDurum::INIT:
            // Donanim sensor kalibrasyonu yap
            break;
        case RobotDurum::CALISMA:
            // Yolu takip et ve eyleyicileri sur
            break;
        case RobotDurum::HATA_DURUMU:
            // Tum PWM cikislarini sifirla ve fren uygula
            break;
        default:
```

Bölüm Özeti ve OOP'ye Köprü

Bu derste incelenen yapılar, Nesne Yönelimli Programlamanın sınıflarına temel oluşturur:

- `struct`, C++ dilinde fonksiyonel kurucu ve metot kazanarak bir sınıfa (*class*) dönüşmüştür.
- Kapsülleme ilkesi ile `public` yapı üyeleri, `class` içerisinde `private` veri gizleme ile korunacaktır.
- `union` ve `bit-fields` gömülü ve mekatronik donanım seviyesinde vazgeçilmez düşük seviyeli araçlardır.
- `enum class`, tip güvenliği sağlayarak sayısal sihirli sayıların (*magic numbers*) önüne geçer.
- Önümüzdeki hafta: OOP'ye Giriş: Sınıflar, Nesneler ve Yaşam Döngüsü.

> **Terim:** Sınıf Değişmezi (Class Invariant), bir sınıf nesnesinin yaşam döngüsü boyunca her zaman geçerli ve doğru kalması gereken dahili durum kuralıdır.

Laboratuvar Çalışması ve Uygulama Yönergeleri

Haftalık 2 saatlik laboratuvar oturumunda aşağıdaki uygulamalar canlı kodlanacaktır:

- **Uygulama 1:** `#pragma pack(1)` ve dolgulu `struct` boyutlarının `sizeof` ile analizi.
- **Uygulama 2:** Bir `union` aracılığıyla 32-bit kayan noktalı sensör verisinin 4 bayta parçalanması.
- **Uygulama 3:** 8-bitlik motor durum kaydedicisinin bit alanları ile modellenmesi ve bayrak kontrolü.
- **Uygulama 4:** `enum class` tabanlı hata yönetim modülü ve güvenli durum makinesi implementasyonu.
- **Değerlendirme:** Kodlar laboratuvar ortamında derlenerek canlı test edilecektir; ev ödevi verilmez.

> **Terim:** Canlı Kodlama (Live Coding), öğrencilerin laboratuvar ortamında derleyici başında birebir uygulayarak gerçekleştirdiği interaktif yazılım pratiğidir.

Teşekkürler

Sorular ve Tartışma

Nesne Yönelimli Programlama • Hafta 6 Marmara Üniversitesi • Mekatronik Mühendisliği Bölümü